

实验 1 中我完成了对 Rust 语言的所有单词的匹配，单词文法参考为 [token - Rust 参考手册 中文版](#)，本次测试我将完成如下的任务：

一、任务概述

1. 用正则表达式表示其中所有能表示的单词，并将无法采用正则表达式表示的单词在本测试报告中逐个列出
2. 按照图 1 DFA 图画法的参考图示方式将实验 1 所需要完成的所有单词的 DFA 图画出

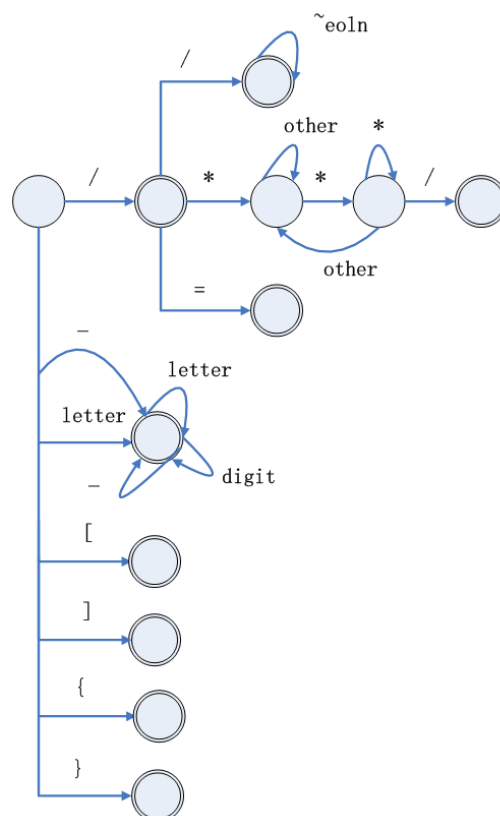


图 1 DFA 图画法的参考图示

3. 运行实验 2 的程序，输入上述要求 1 中得到的正则表达式，并将响应的输出结果（最小化的 DFA）进行截图，通过比较上述要求 2 中所画的 DFA 和实验 2 中输出的结果，写出我所实现的试验 2 的测试结果

二、任务 1——用正则表达式表示所有单词

由 [token - Rust 参考手册 中文版](#) 可知，所有的能用正则表达式表示的单词的正则表达式如下所示：

注意：由于实验 2 个人所编写的单词匹配格式和参考手册的有所不同，因此所写出的正则表达式也和参考手册的略有差异，但能够正确实现其正则表达式的功能

整数字面量:

binDigit=[0-1]

octDigit=[0-7]

decDigit=[0-9]

hexDigit=[0-9a-fA-F]

integerSuffix=u8 | u16 | u32 | u64 | u128 | usize | i8 | i16 | i32 | i64 | i128 | isize

binLiteral=0b((binDigit|_)*)((binDigit))((binDigit|_)*)

octLiteral=0o((octDigit|_)*)((octDigit))((octDigit|_)*)

decLiteral=((decDigit|_)*)((decDigit))

hexLiteral=0x((hexDigit|_)*)((hexDigit))((hexDigit|_)*)

_integerLiteral=(binLiteral|octLiteral|decLiteral|hexLiteral)integerSuffix?

浮点型字面量:

decDigit=[0-9]

decLiteral=((decDigit|_)*)((decDigit))

floatExponent=((e|E)(\+|-))?((decDigit|_)*)

floatSuffix=f64|f32

_floatLiteral=(decLiteral(\.))|((decLiteral)(floatExponent))|((decLiteral)(\.)decLiteral)
(floatExponent ?)|((decLiteral)(\.)decLiteral)?(floatExponent)?(floatSuffix))

字符字面量:

octDigit=[0-7]

decDigit=[0-9]

hexDigit=[0-9a-fA-F]

notspecialchar=\+|\-|*|/|\?|&&|<<|>>|<=<|>=>|=|=|=|!=|>|<|>->|\{|\}|\(|\)|&|=|+|=|-
=|*=|/=|%|=|^=|\.|\\||=|\"|\"|[0-9a-zA-Z]|~|\,

quoteEscape=(\\)(\"")

asciiEscape=(\x)(octDigit)(hexDigit)\n\r\t\\0\,

unicodeEscape=(\u)\{(hexDigit_*)\}

_charLiteral=(\')(notspecialchar|quoteEscape|asciiEscape|unicodeEscape)(\')

注意：notspecialchar 表示的是除了单引号、空格、换行符、制表符外的所有字符

字符串字面量：

notspecialchar=\+|\-|*|/|\?|&&|<<|>>|<=<|>=>|=|=|=|!=|>=<=|->|\{|\}|\(|\)|&|=|+=|-
=|*=|/=|%|=|^=\.|\|||=\'|[0-9a-zA-Z]\~|\t|\n|\r|\,

stringContinue=(\\)(\n)

quoteEscape=(\')(\"')

asciiEscape=(\x)(octDigit)(hexDigit)\n\r\t\\0

unicodeEscape=(\u)\{(hexDigit_*)\}

_stringLiteral=(\")(notspecialchar|quoteEscape|asciiEscape|unicodeEscape|stringContinue
)\"')

注意：notspecialchar 表示的是除了双引号、空格、反斜杠外的所有字符

原生字符串字面量：

notspecialchar=\+|\-|*|/|\?|&&|<<|>>|<=<|>=>|=|=|=|!=|>=<=|->|\{|\}|\(|\)|&|=|+=|-
=|*=|/=|%|=|^=\.|\|||=\'|[0-9a-zA-Z]\~|\t|\,

rawStringContent=(\")(notspecialchar(\")(\\#)rawStringContent(\\#)

_rawStringLiteral=rrawStringContent

注意：notspecialchar 表示的是除了孤立回车（即\r 后不跟\n 的情况）外的所有字符

其中，rawStringContent 行的不是正规语言，不存在 DFA

字节字面量：

hexDigit=[0-9a-fA-F]

asciiForChar=\+|\-|*|/|\?|&&|<<|>>|<=<|>=>|=|=|=|!=|>=<=|->|\{|\}|\(|\)|&|=|+=|-
=|*=|/=|%|=|^=\.|\|||=\'|[0-9a-zA-Z]\~|\'

`byteEscape=(\x)hexDigithexDigit|(\n)|(\r)|(\t)|(\0)`

`_byteLiteral=b`'(asciiForChar|byteEscape)`'`

注意：asciiForChar 表示的是除了逗号、反斜杠、单引号、回车符、制表符外的所有字符

字节串字面量：

`hexDigit=[0-9a-fA-F]`

`asciiForString=\+|\-|\\*|\/|\\?|&&|<<|>>|<=>|>=|==|!=|>=|<=|>->|\\{\\}|\\(|\\)|&=|+=|-=|*=|/=|%=|^=|\\.\\.\\.|=|[0-9a-zA-Z]|\\~|\\t|\\n|\\r|\\'`

`byteEscape=(\x)(hexDigit)(hexDigit)|(\n)|(\r)|(\t)|(\0)`

`stringContinue=(\\)(\\n)`

`_byteStringLiteral=b`"(asciiForString|byteEscape|stringContinue)`"`

注意：asciiForString 表示的是除了逗号、双引号、反斜杠、空格外的所有字符

原生字节串字面量：

`rawByteStringContent=(\\")ascii*(\\")|(\#)rawByteStringContent(\#)`

`ascii=\+|\-|\\*|\/|\\?|&&|<<|>>|<=>|>=|==|!=|>=|<=|>->|\\{\\}|\\(|\\)|&=|+=|-=|*=|/=|%=|^=|\\.\\.\\.|=|[0-9a-zA-Z]|\\~|\\t|\\n|\\r|\\'|\\.|\\\"`

`_rawByteStringLiteral=brrawByteStringContent`

注意：ascii 表示的是所有 ASCII 字符

其中，rawByteStringContent 行的不是正规语言，不存在 DFA

布尔字面量：

`true=true`

`false=false`

`booleanLiteral=true|false`

目前参考手册中的所有字面量均可以用正则表达式表示出来，无不能用正则表达式表

示的单词

三、任务 2——画出单词的 DFA 图

按照图 1 DFA 图画法的参考图示方式，各单词的 DFA 图绘制如下：

整数字面量：

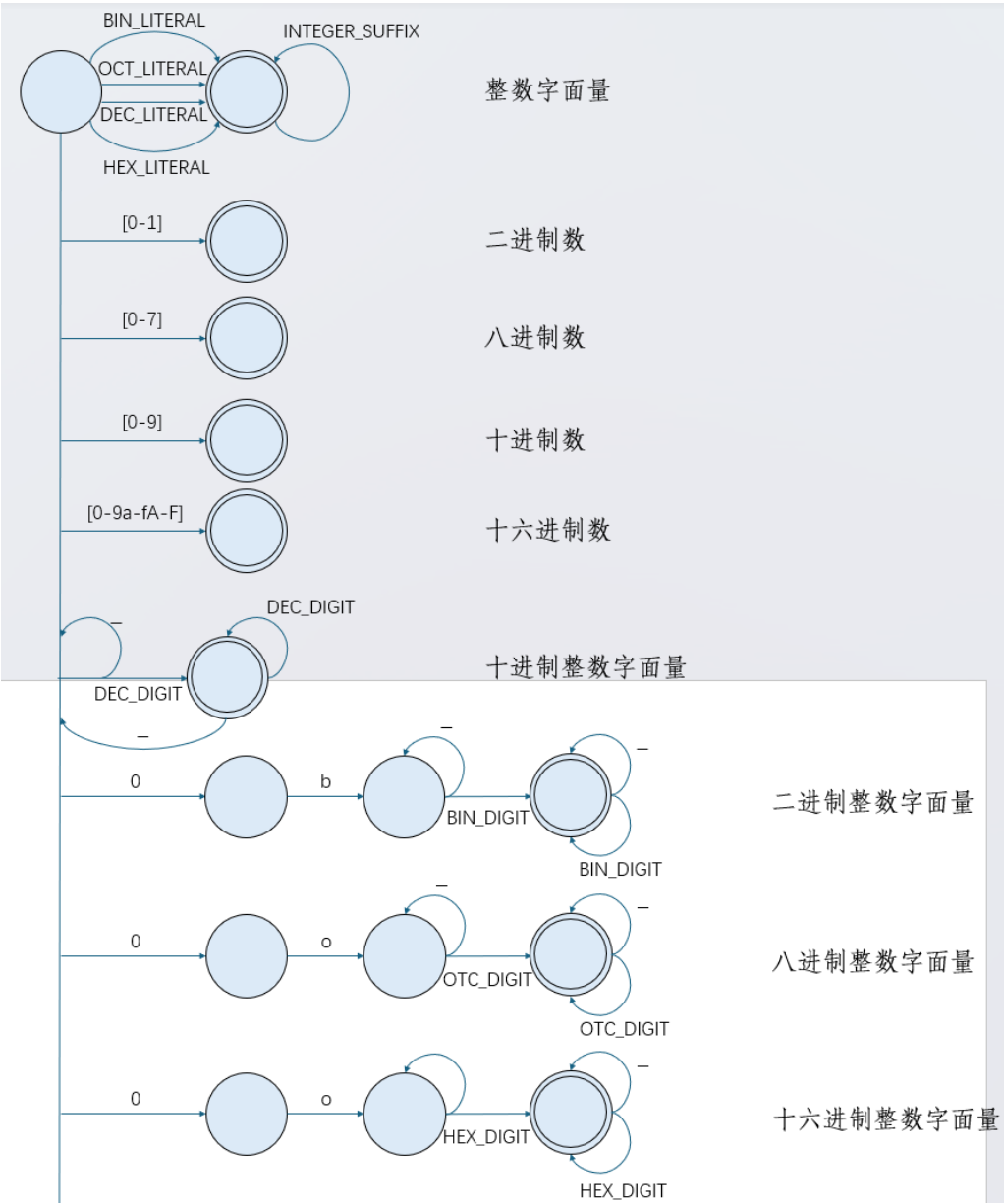
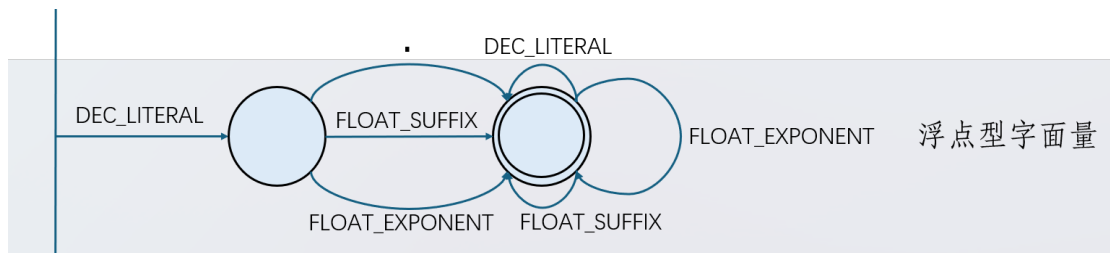
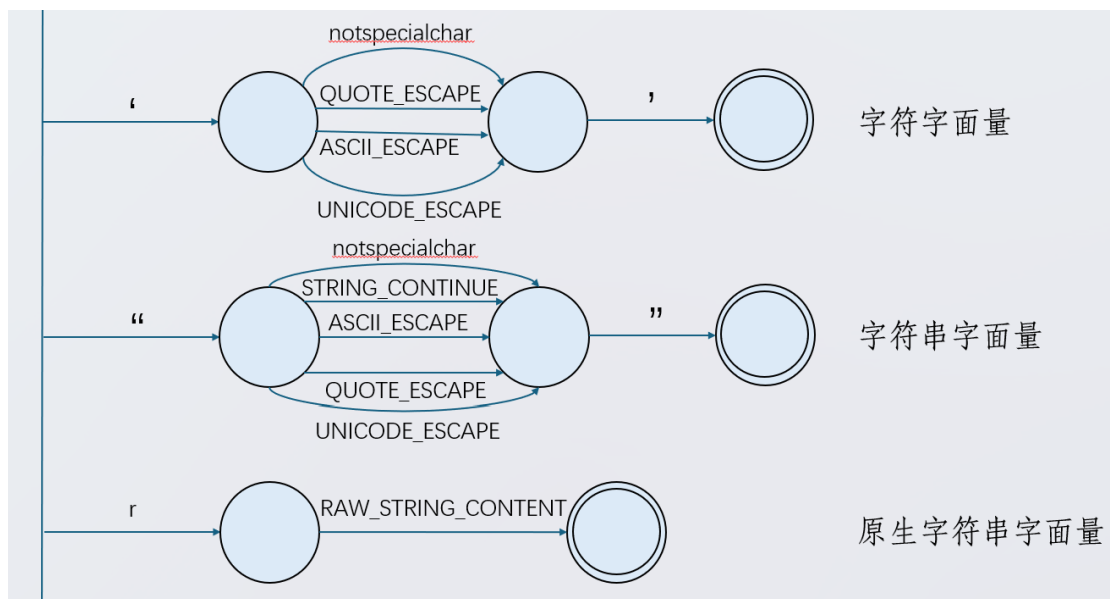


图 2 整数字面量的 DFA 图

浮点型字面量：



字符字面量、字符串字面量和原生字符串字面量：



字节字面量、字节串字面量和原生字节串字面量:

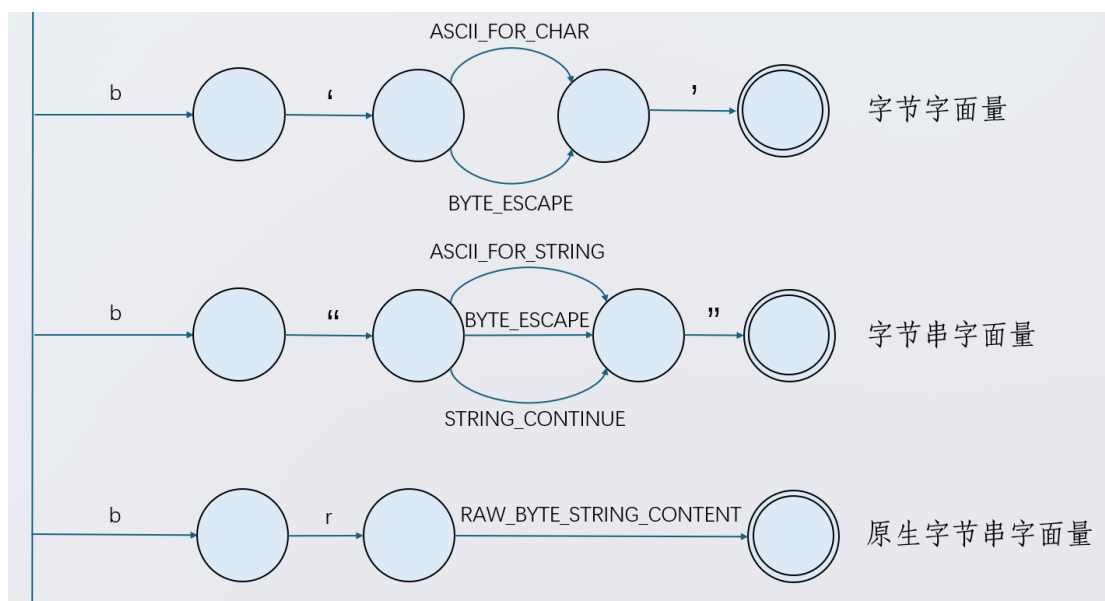


图 5 字节字面量、字节串字面量和原生字节串字面量的 DFA 图

布尔字面量:

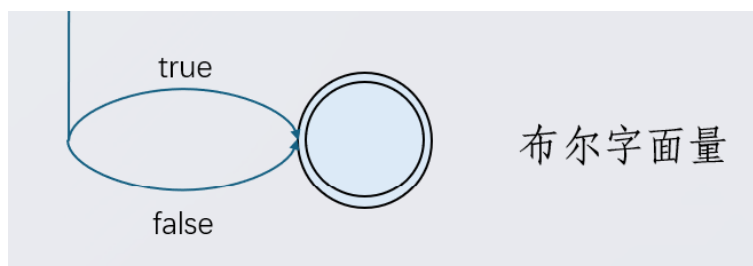


图 6 布尔字面量的 DFA 图

关键字和注释:

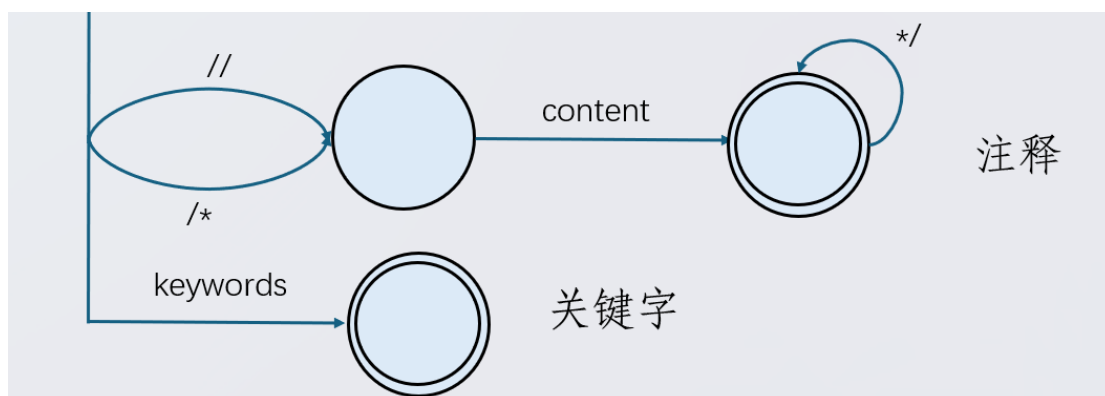


图 7 关键字和注释

三、任务 3——实验 2 程序验证测试

接下来，将刚才的正则表达式输入到实验 2 的系统中，将其转换为最小 DFA 图，实验的结果截图如下：

注意：由于实验 2 中对 DFA 图进行最小化处理时，合并得到的状态是用合并前的各个状态的序号共同表示（如状态 1 和状态 2 合并后的状态用“1 2”表示，而不是使用一个全新的序号表示），因此会和我在任务 2 中画的最小 DFA 图有些许的出入，但也能正确的表示出状态转换的关系

整数字面量和浮点型字面量：

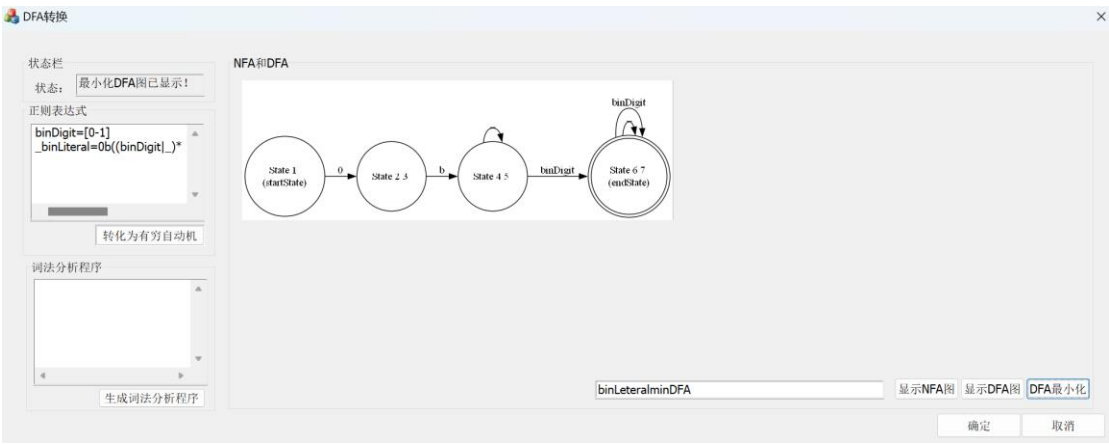


图 7 二进制整数字面量的最小 DFA 图

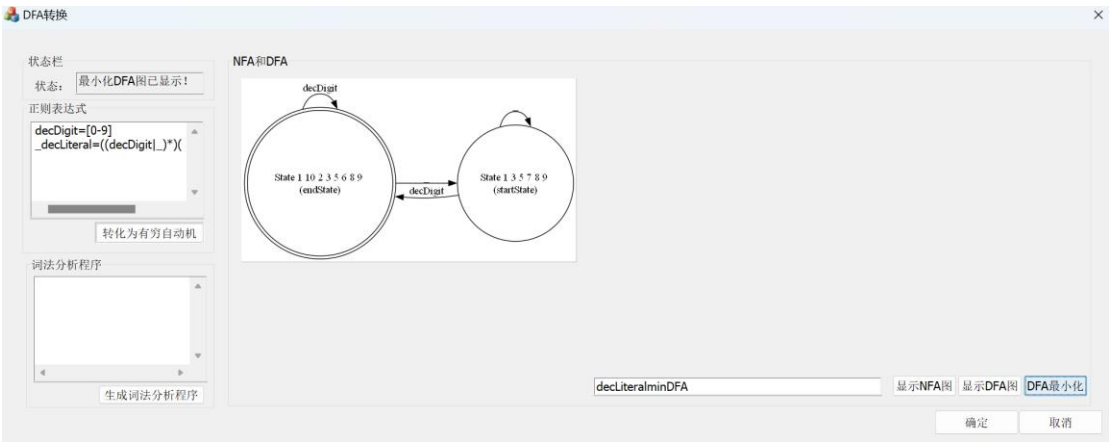


图 8 十进制整数字面量的最小 DFA 图

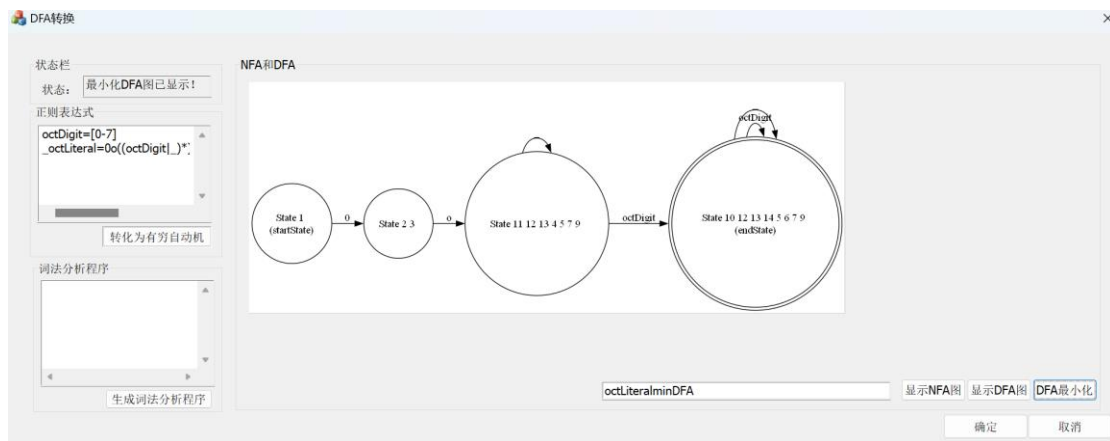


图 9 八进制整数字面量的最小 DFA 图

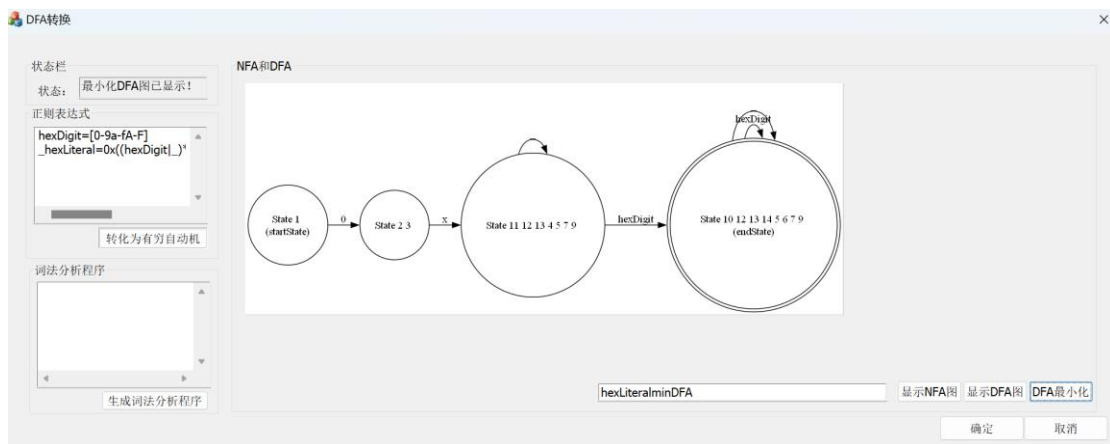


图 10 十六进制整数字面量的最小 DFA 图

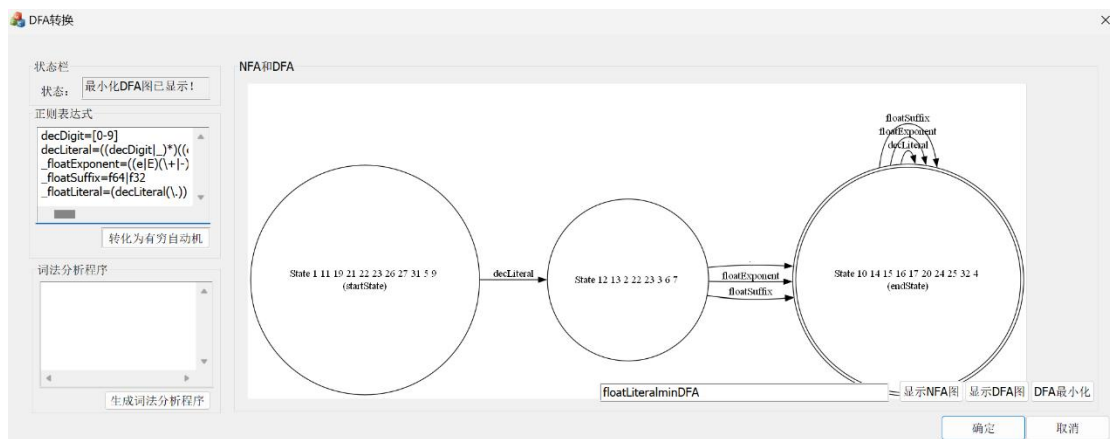


图 11 浮点型字面量的最小 DFA 图

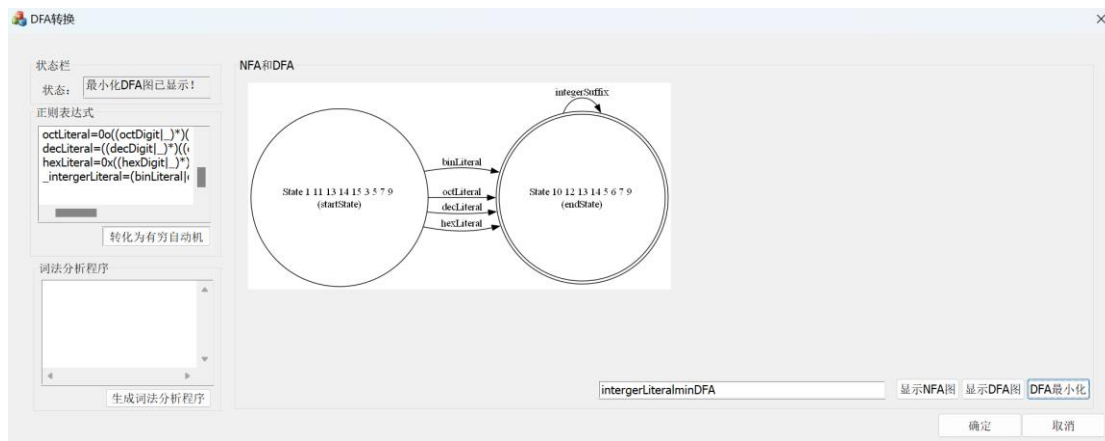


图 12 整数字面量的最小 DFA 图

字符字面量、字符串字面量、原生字符串字面量:

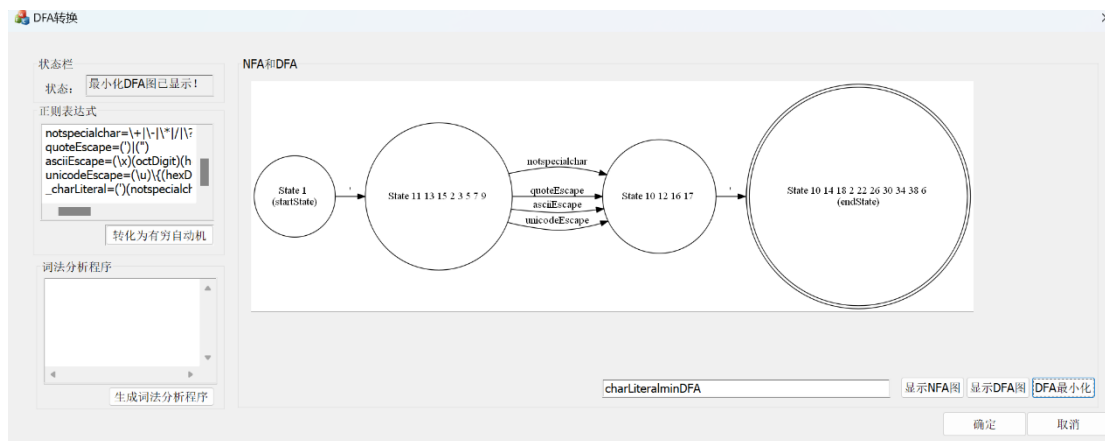


图 13 字符字面量的最小 DFA 图

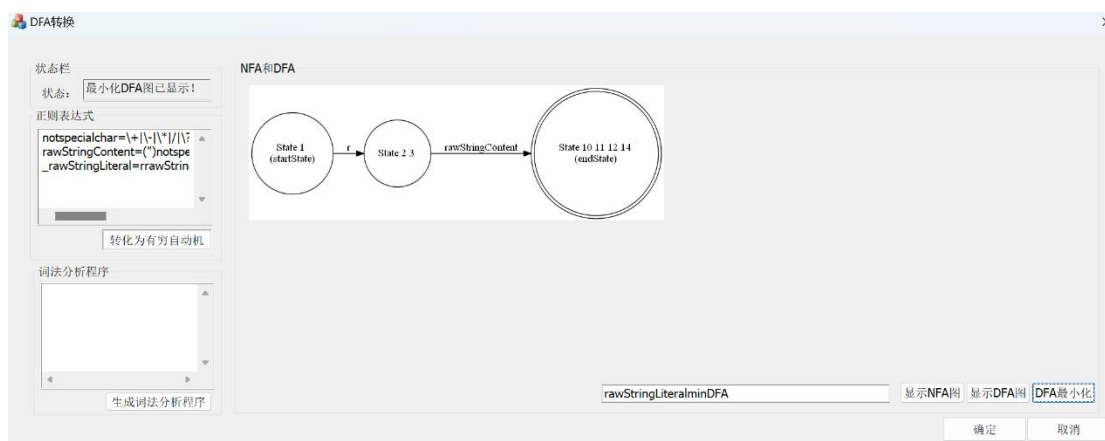


图 14 原生字符串字面量的最小 DFA 图

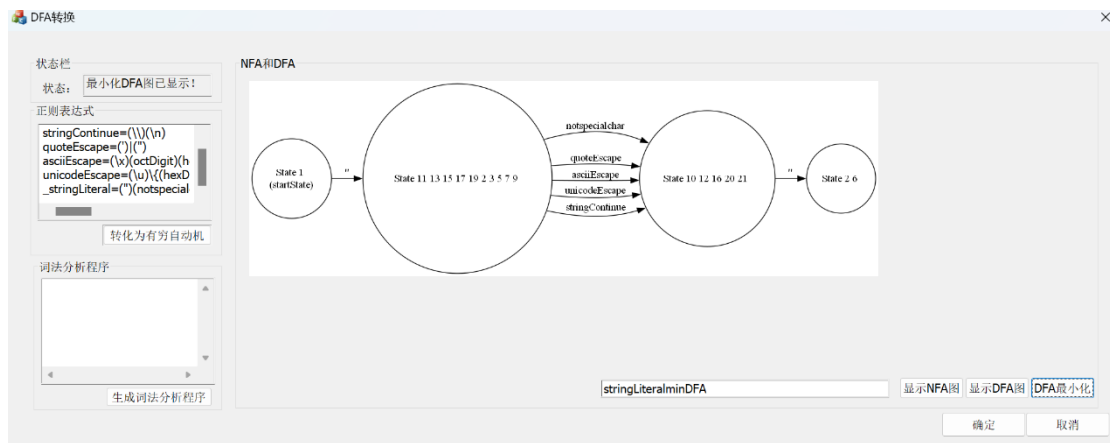


图 15 字符串字面量的最小 DFA 图

字节字面量、字节串字面量、原生字节串字面量:

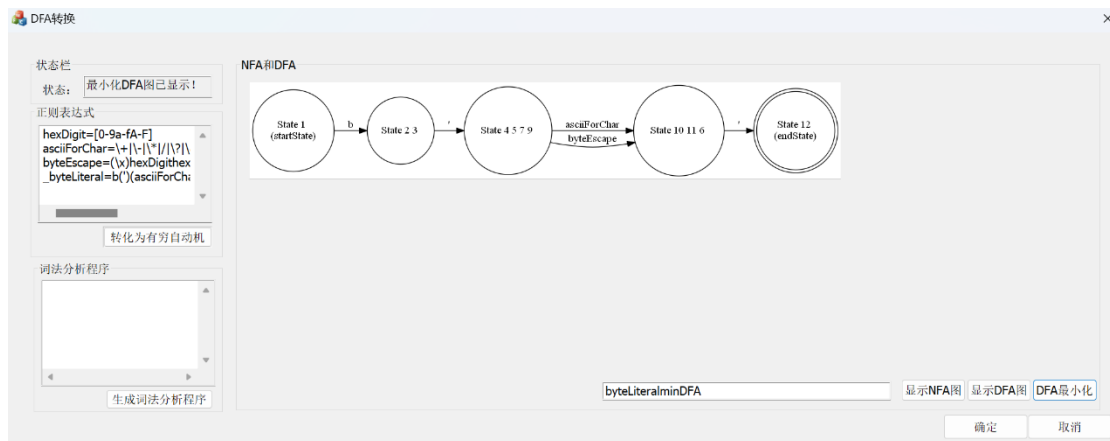


图 16 字节字面量的最小 DFA 图

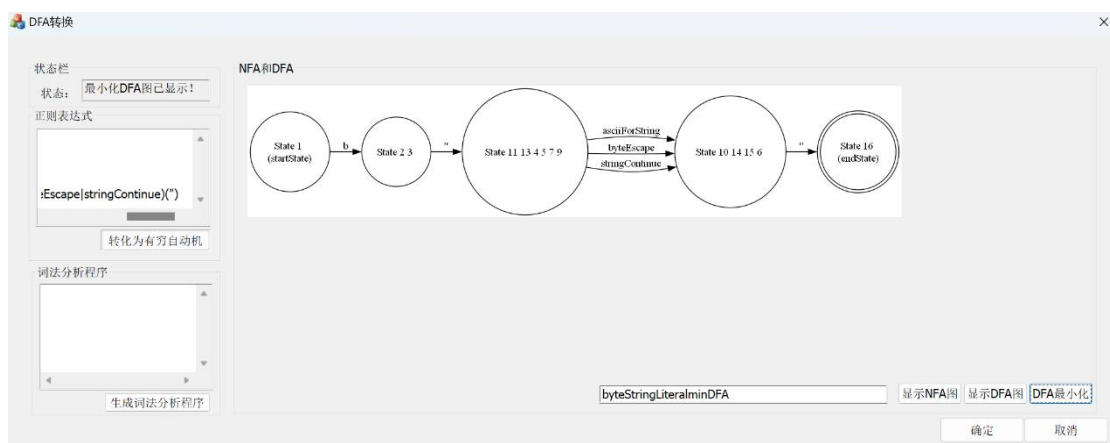


图 17 字节串字面量的最小 DFA 图

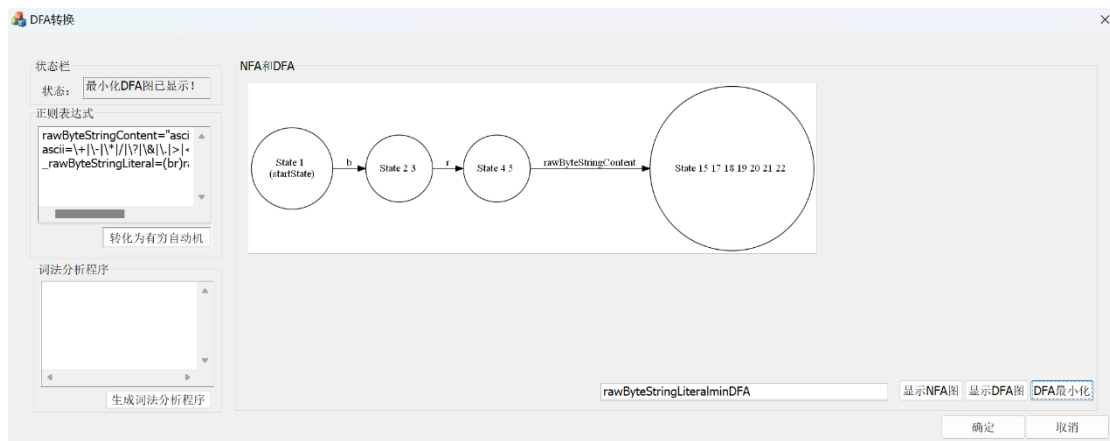


图 18 原生字节串字面量的最小 DFA 图

布尔字面量：

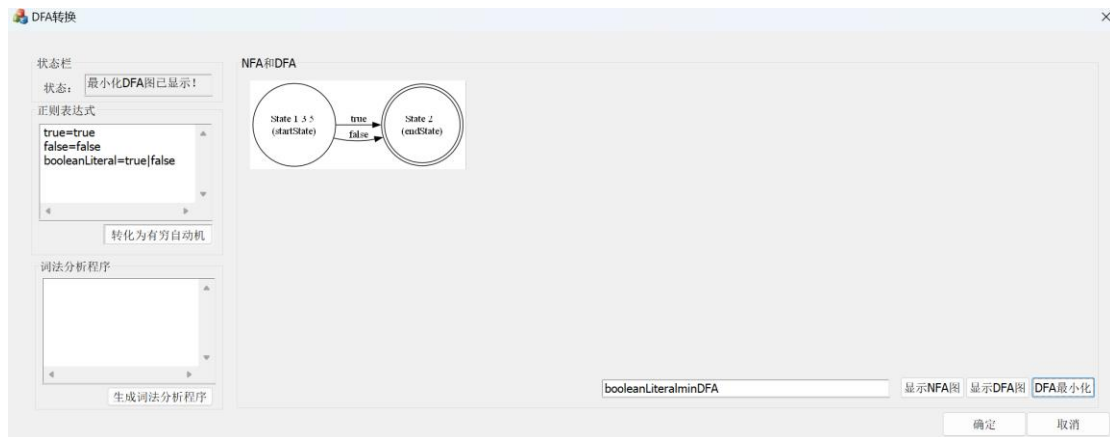


图 19 布尔字面量的最小 DFA 图

经过对比，任务 2 中我手绘的最小 DFA 图和实验 2 的程序生成的最小 DFA 图基本一致

四、附加项——实验 2 之前的测试数据及结果

测试用例 1：

digit=[0-9]

_num100=(\+|-)?digit+

测试用例 2:

letter=[A-Da-d]

测试用例 3:

x=a*

y=b*

z=a(a|b)*

测试用例 4:

token=a(b|c)d

测试用例 5:

digits=[0-9]

letters=[A-Za-z]

_ID101=letter(letter|digit)*

测试用例 6:

_specail200S=\+|-|*|/|=|<|<=<<

测试结果

